🧭 Guide de l'animateur

Ce guide complète les diaporamas (qui contiennent aussi des **notes animateur** en commentaire) et les fiches élèves.

---

1. La philosophie du cours

Un seul projet qui grandit

Chaque séance ajoute **une fonctionnalité visible** au même jeu. La notion de programmation arrive **parce que le jeu en a besoin** :

- on veut 5 météorites → on a besoin d'une **liste** et d'une **boucle** ;
- on veut que le vaisseau ne sorte pas → on a besoin d'une **condition**.

Toujours présenter la notion **après** avoir posé le problème (« Et si on voulait 100 météorites ? »).

Une aide qui diminue petit à petit

Séances	Forme de l'aide
1 – 3	trous ___ à remplacer, lignes à recopier en adaptant
4 – 6	commentaires `>>> A TOI` qui disent **quoi** faire, la fiche donne une partie du **comment**
7 – 8	commentaires qui décrivent le but, l'enfant écrit des blocs entiers
9 – 10	**missions** : l'enfant cherche lui-même où placer le code

Trois règles à répéter

1. **On tape le nouveau code**, on ne le copie-colle pas.
2. **On teste souvent** : F5 après chaque petit changement.
3. **Une erreur n'est pas un échec**, c'est un indice. On lit le message rouge.

La routine « Prédis, lance, observe »

Chaque séance contient au moins une diapo **« 🤔 Que va faire ce code ? »**. Faites toujours **prédire avant de lancer** (à voix haute ou sur la fiche). C'est ce moment qui fait le plus progresser.

---

2. Déroulé type (1 h 30)

Durée	Moment	Conseils
5 min	Démarrage	Les installations tournent pendant le rappel.
10 min	Rappel	Quiz oral rapide. Faites jouer 30 s à la version de la séance précédente.
15 min	Notion du jour	Une image du quotidien, une activité débranchée si prévue, une diapo « Que va faire ce code ? ».
45 min	Atelier	Vous circulez. Les enfants suivent la fiche, étape par étape.

10 min	Défis bonus	Pour ceux qui ont fini. Les autres terminent l'atelier : c'est le plus important.
5 min	Sauvegarde + bilan	Chaque enfant dit une chose qu'il a apprise.

---

3. Comment aider un enfant bloqué (sans donner la réponse)

Dans l'ordre, essayez :

1. **« Qu'est-ce que tu voulais qu'il se passe ? Et qu'est-ce qui se passe ? »**
2. **« Lis-moi le message d'erreur. Quelle ligne ? »** (Dans IDLE, la ligne fautive est surlignée.)
3. **« Où dans ton code on a déjà fait quelque chose qui ressemble ? »** (ex. : `aller_a_gauche` pour écrire `aller_a_droite`)
4. **Montrez la ligne** sans la corriger : « Regarde bien cette ligne, il manque un caractère. »
5. En dernier recours, **tapez avec lui**, et faites-lui expliquer ce que fait la ligne.

> 💡 *La règle des 3 avant moi : avant d'appeler l'animateur, l'enfant relit sa fiche, regarde l'affiche du détective des bugs, et demande à un voisin.*

Avec 6 enfants, gardez un œil sur ceux qui n'appellent jamais à l'aide : ils sont parfois bloqués en silence.

---

4. Les erreurs les plus fréquentes

Erreur	Cause probable	Ce qu'on dit à l'enfant
`NameError: name '___'`	un trou pas rempli	« Il reste un trou à remplir. »
`NameError` (autre mot)	faute de frappe, majuscule	« Compare lettre par lettre avec l'endroit où tu l'as créé. »
`SyntaxError`	`:` oublié, `)` ou `"` manquant	« Regarde la fin de la ligne, et la ligne juste au-dessus. »
`IndentationError`	décalage incohérent	« Les lignes d'un même bloc doivent commencer à la même colonne. »
`UnboundLocalError`	`global` oublié	« Si tu changes une variable du jeu dans une fonction, il faut `global`. »
`TypeError: can only concatenate str`	`"Score : " + score`	« Transforme le nombre en texte avec `str()`. »
Les touches ne marchent pas	la fenêtre du jeu n'a pas le focus	« Clique une fois dans la fenêtre du jeu. »
La fenêtre est figée	erreur dans la boucle de jeu	« Regarde le Shell d'IDLE, il y a un message rouge. »
Code après `ecran.mainloop()`	ces lignes ne sont jamais exécutées	« `mainloop` doit toujours rester tout en bas. »

---

5. Notes séance par séance

Séance 1 — Bonjour Python !

- **Préparez** : testez le kit sur un PC de la salle avant la séance. Ce kit n'a pas de `depart.py` : IDLE s'ouvre sur le Shell `>>>`, c'est voulu.
- **Ne mettez pas** `carte_pilote_solution.py` dans le kit.
- **Commencez par montrer le jeu final** (1 minute) : c'est la motivation pour tout le cycle.
- **Déroulé conseillé** : accueil + jeu final 5 min · partie 1 « l'informatique » 15 min · partie 2 « l'IA » 10 min · Shell, print, variables, input 20 min · carte de pilote 25 min · sauvegarde 5 min. La tortue (étape 5) passe en bonus ; si personne n'y arrive, montrez-la au tableau (2 min) car la séance 2 s'appuie dessus.
- **Parties 1 et 2 (informatique et IA)** : écrites pour être comprises dès 6–8 ans. Faites participer : objets cachés, sandwich dans l'ordre, jeu du robot, jeu du mot suivant. Évitez le vocabulaire technique au-delà de « programme », « langage », « IA ».
- **IA** : insistez sur les 3 messages de prudence (elle peut se tromper, on ne donne pas d'infos personnelles, on l'utilise avec un adulte). La diapo Mistral est volontairement factuelle : une entreprise française, son nom (le vent), son assistant Le Chat, des IA partagées.
- **Activités débranchées** : « le robot » (un enfant obéit à la lettre aux instructions des autres), puis une boîte étiquetée « prenom » pour les variables.
- **Shell vs éditeur** : le Shell répond ligne par ligne, l'éditeur garde un programme entier qu'on lance avec F5. Montrez les deux.
- **Piège** : oublier les guillemets → `NameError`. Faites l'erreur exprès au tableau : c'est le premier « bug » du cycle, dédramatisez.
- **Piège** : répondre « onze » à la question de l'âge → `ValueError`. Bonne occasion d'expliquer `int()`.
- `print("Bonjour", prenom)` avec des **virgules** suffit pour cette séance ; la concaténation avec `+` et `str()` arrive à la séance 5.

Séance 2 — Prends les commandes

- C'est la séance la plus **dense** (fenêtre + vaisseau + clavier). Si le temps manque, gardez les défis pour la séance 3.
- **Point clé** : `x = x - vitesse` n'est pas une équation de maths. Reprenez la boîte de la séance 1 : « range dans la boîte ».
- **Piège** : les couleurs sont en anglais. Donnez une liste : `black`, `navy`, `midnightblue`, `cyan`, `orange`, `lime`, `hotpink`, `gold`, `white`.
- **Piège** : le `y` est positif **vers le haut**.
- **Piège** : l'indentation. Montrez qu'IDLE décale tout seul après les `:`.
- Faites le lien avec le jeu Ruby : là-bas ils utilisaient des commandes, ici ils les **fabriquent**.

Séance 3 — Reste dans l'écran

- **Piège** : `=` au lieu de `==` ou `<`. Donnez le moyen mnémotechnique « un `=` range, deux `==` comparent ».
- **Question fréquente** : « Pourquoi 380 et pas 400 ? » → la moitié du vaisseau dépasserait.

Séance 4 — Il pleut des étoiles

- **Image** : le flipbook / dessin animé.
- **Point clé** : `ontimer` relance la fonction : c'est une boucle « cachée ». Cela prépare la séance 6.
- `ecran.tracer(0)` + `ecran.update()` : présentez-les comme « le bouton qui rend le jeu fluide », sans entrer dans le détail.

- **Piège** : oublier `boucle_de_jeu()` en bas → rien ne bouge.

Séance 5 — Attrape-les toutes

- Souvent la séance de **reprise après la Toussaint** : prévoyez un grand rappel.
- `global` : ne pas théoriser. « Pour changer une variable du jeu dans une fonction, on écrit `global` au début. »
- **Testez `str()` dans le Shell** d'IDLE devant eux : `"Score : " + 5` → erreur, `"Score : " + str(5)` → ok.

Séance 6 — Pluie de météorites ⭐ séance clé

- **Activité débranchée indispensable** : « le train ». `meteorites = []` = train vide ; `append` = un enfant monte au bout ; `for m in meteorites` = l'animateur passe devant chaque wagon et lui dit « descends d'un pas ».
- **Effet waouh** : réécrire le carré de la séance 1 avec `for i in range(4)` .
- **Piège** : `append` pas indenté dans la boucle → une seule météorite dans la liste.
- Attendez-vous à plus de demandes d'aide : c'est normal.

Séance 7 — Game over

- Le fichier de départ a été **réorganisé** (fonctions `replacer_en_haut` , `afficher_infos` , `afficher_message`). Prenez 5 minutes pour le parcourir avec eux : « un programmeur range son code ».
- **Image** : la recette « jus de ___ » pour expliquer le paramètre.
- **Piège** : `true` au lieu de `True` .
- **Piège** : oublier de replacer la météorite après la collision → toutes les vies perdues d'un coup.

Séance 8 — Les lasers

- **Activité débranchée** : reprendre le train (on monte avec Espace, on descend quand on sort de l'écran).
- `lasers.copy()` est déjà écrit : si on vous demande, « on ne retire pas un wagon pendant qu'on passe devant ». Pas besoin d'aller plus loin.
- **Piège** : `"space"` s'écrit en minuscules.

Séance 9 — Mon style

- **Préparez** : vérifiez que les fichiers `.gif` et `.wav` sont bien dans le kit, et que les PC ont du son (casques ?).
- Pour dessiner leurs sprites : **piskelapp.com** (Internet requis). Sinon, Paint en GIF sur fond bleu nuit (pas de transparence).
- **Piège** : `register_shape` après `shape` → erreur « no shape named ».
- **Piège** : oublier `niveau` et `vitesse_chute` dans les lignes `global` .

Séance 10 — Finitions et démo

- **Organisation** : 45 min de finitions, 20 min de tests croisés et de préparation, 25 min de démo (ou une séance de démo à part).
- **Démo** : chaque enfant présente 2 minutes avec sa fiche, puis un parent essaie de battre son record.
- Remettez les **diplômes** (`seance-10/diplome.md`).
- Donnez aux parents la consigne pour rejouer à la maison (fin de la fiche élève).

---

6. Gérer les écarts de niveau

- **Ceux qui vont vite** : défis bonus ⭐⭐⭐, puis « aide un camarade en lui expliquant (sans toucher son clavier) ».
- **Ceux qui sont lents** : ils doivent finir l'**atelier**, pas les défis. Le fichier de la séance suivante contient de toute façon la solution complète : personne ne prend de retard d'une séance à l'autre.
- **Ceux qui étaient absents** : ils repartent du `depart.py` de la séance du jour. Résumez-leur en 2 minutes ce qui a été ajouté.

---

7. Vérifier le code avant une séance

Toutes les solutions sont dans `seance-XX/code/solution.py`. Pour vérifier qu'elles fonctionnent toutes :

```
python outils/tester_solutions.py
```

Le script lance chaque solution, simule des touches et des collisions, et signale toute erreur.